MOORE MACHINE

Moore machine is finite Automata (FA) with output. A moore machine is an FSM whose output depends only on the present state.

Tuples of Moore Machine

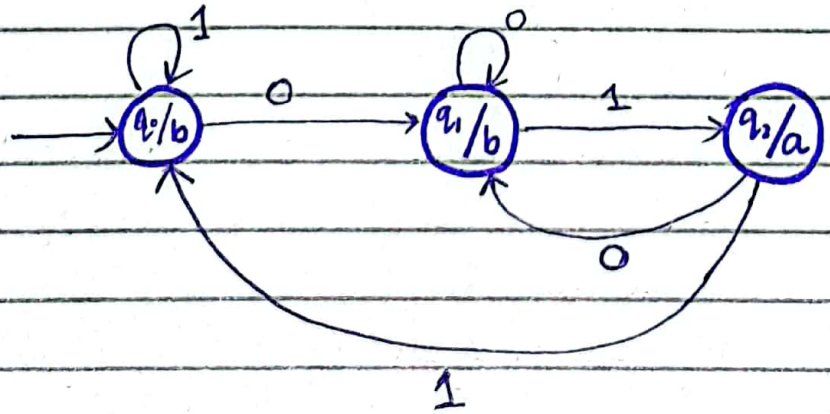
Moore machine has 6-tuples $(Q, \Sigma, \delta, q_0, F)$, where

1- Q is a finite set called the states (e.g. $q_0, q_1, q_2 \ldots, q_n$)
2- Σ is finite set called the alphabet (e.g. $\Sigma = \{a, b\}$)
3- δ transition / movement : $Q \times \Sigma \rightarrow Q$
4- q_0 is the start state (initial state)
5- O is a finite set of symbols called the output alphabet
6- λ is the output transition function where $\lambda : Q \rightarrow O$

Examples :

i) Construct a moore machine that prints 'a' whenever the sequence '01' is encoutered in any input binary string

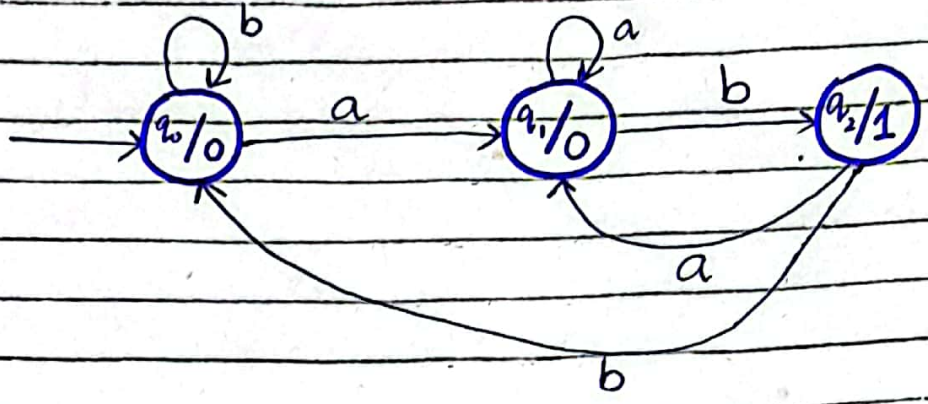
$\Delta = \{a, b\}$
$\Sigma = \{0, 1\}$



ii) Construct a moore machine that prints '1' whenever the sequence 'ab' is encountered in any input binary string

$\Delta = \{0, 1\}$

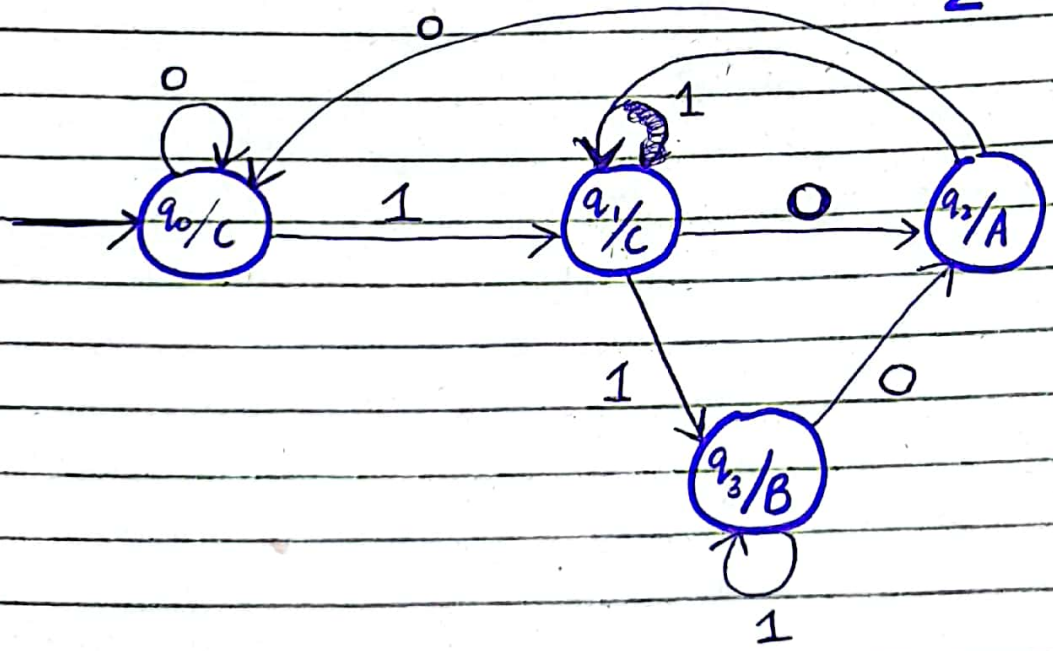
$\Sigma = \{a, b\}$



iii) Construct a moore machine that takes set of all strings {0,1} and produces 'A' as output if input ends with '10' or produces 'B' as output if input ends with '11', otherwise 'C'

$\Delta = \{A, B, C\}$

$\Sigma = \{0, 1\}$



String: 10101100

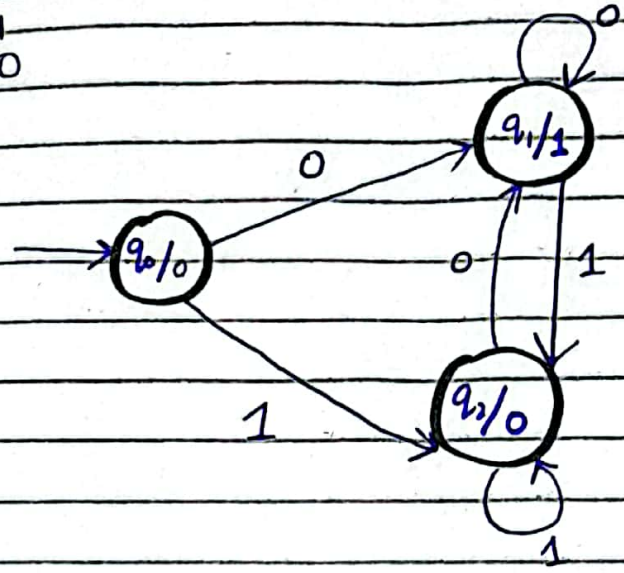
output: CBACBACA

iv) Design a moore machine that produces 1's compliment of any binary input string

binary	1's
0	1
1	0

$\Delta = \{0, 1\}$

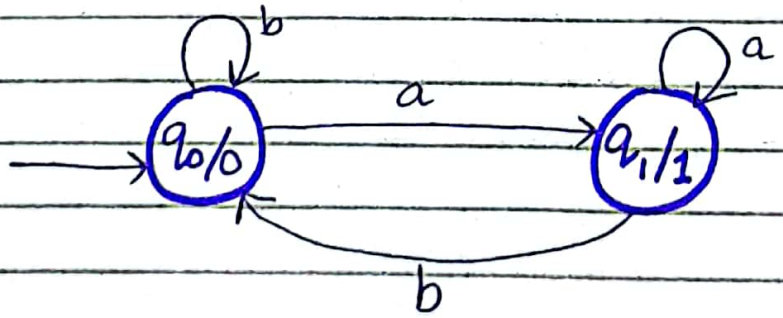
$\Sigma = \{0, 1\}$



string: 1 1 0
output: 0 0 0 1

string: 1010
output: 00101

v) Construct a moore machine that takes all the strings of **a's & b's** as input and counts the no. of a's in the input string in terms of **1s**



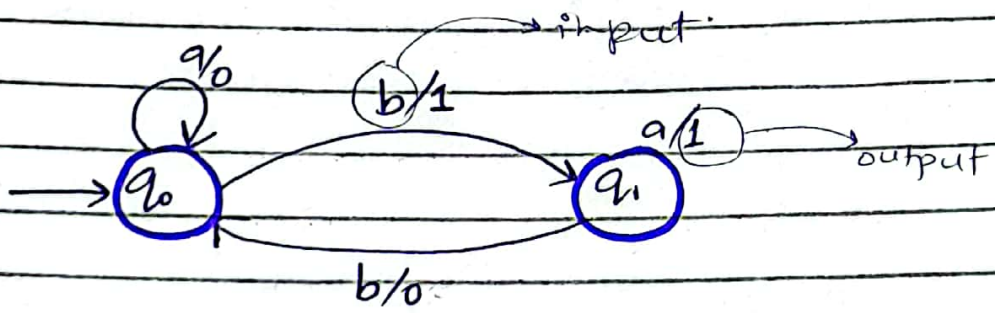
MEALY MACHINE

Mealy machine is a type of finite state machine (FSM) that produce output based on the current state and input symbol.

A mealy machine is a 6-tuple $(Q, \Sigma, \Delta, \delta, q_0, F)$

- Q is a finite set of states
- Σ is a finite set of input symbols
- $\Delta \to O$ is a finite set of output symbols
- δ Transition function
- λ output function
- q_0 is the initial state.

$$\lambda : Q \times \Sigma \to \Delta$$

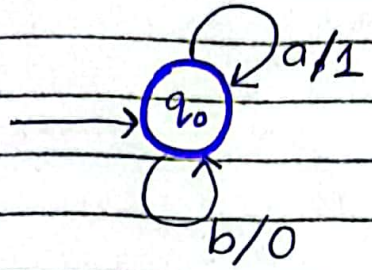


mealy	moore
input = n	input = n
output = n	output = n+1

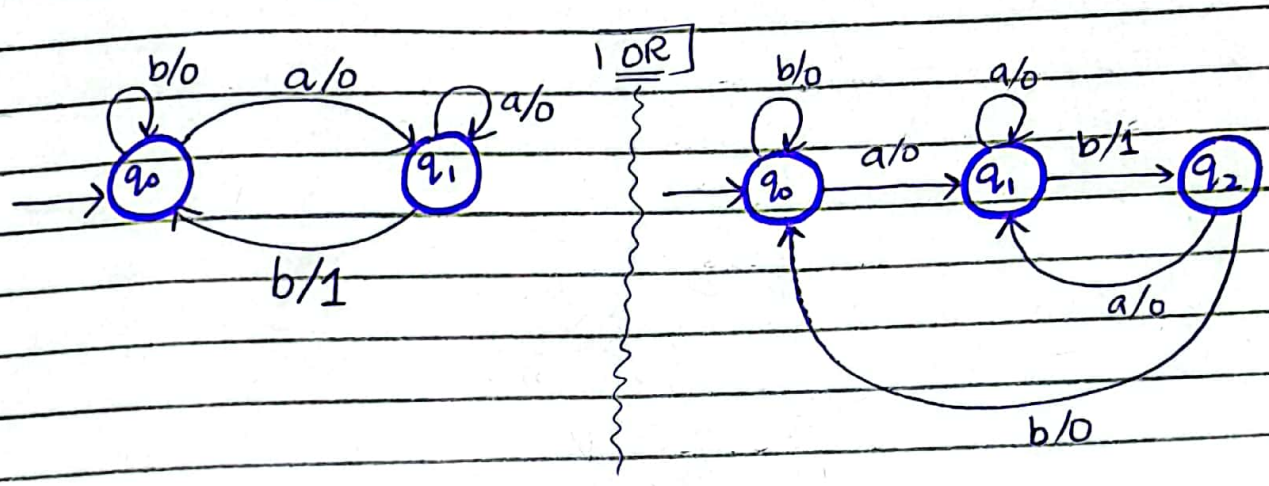
i) construct a mealy machine where at 'a' output is '1' and at 'b' is '0'

$\Delta = \{0, 1\}$

$\Sigma = \{a, b\}$



ii) construct a mealy machine that prints '1' whenever the substring 'ab' is countered

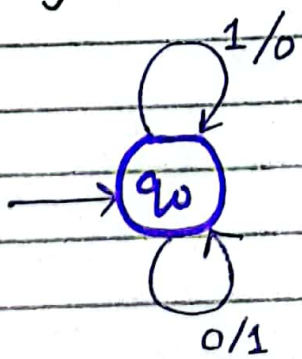


iii) Design a moore machine that produces 1's compliment of any binary input string

$\Delta = \{0, 1\}$

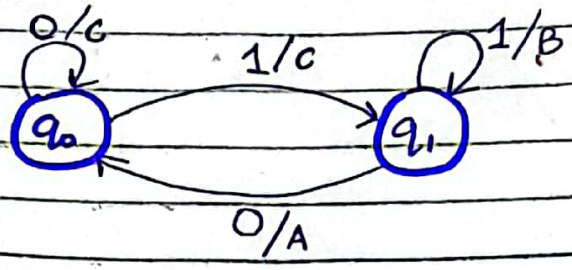
$\Sigma = \{0, 1\}$

binary	1's compliment
0	1
1	0



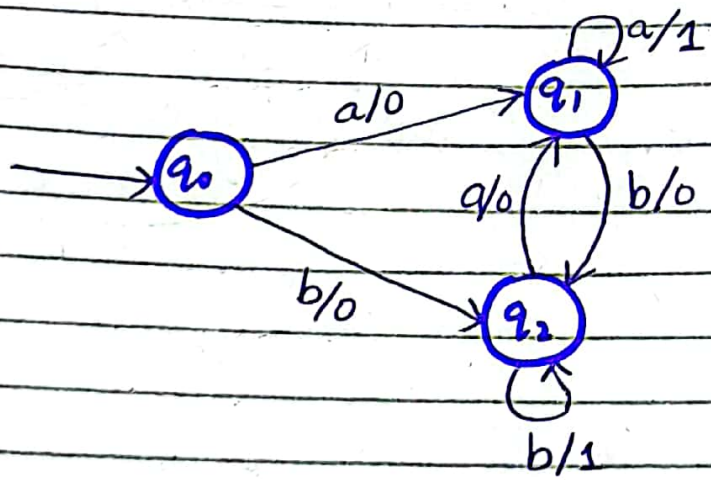
iv) Construct a mealy machine that takes all string over {0,1} and produces 'A' as output if input ends with '10' or 'B' if input ends with '11' otherwise 'C'.

$\Delta = \{A, B, C\}$
$\Sigma = \{0, 1\}$

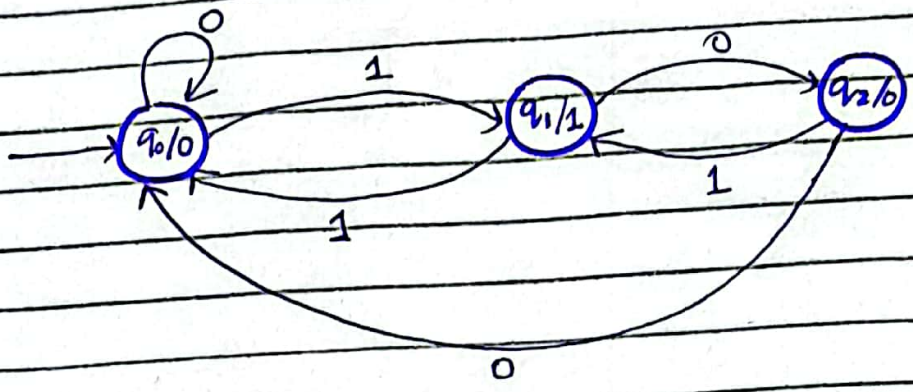


v) a mealy machine accepting the language consisting of string from $\Sigma = \{a, b\}$ and the string should end with either 'aa' or 'bb' having an output of '1'

$\Delta = \{0, 1\}$
$\Sigma = \{a, b\}$



MOORE TO MEALY Conversion

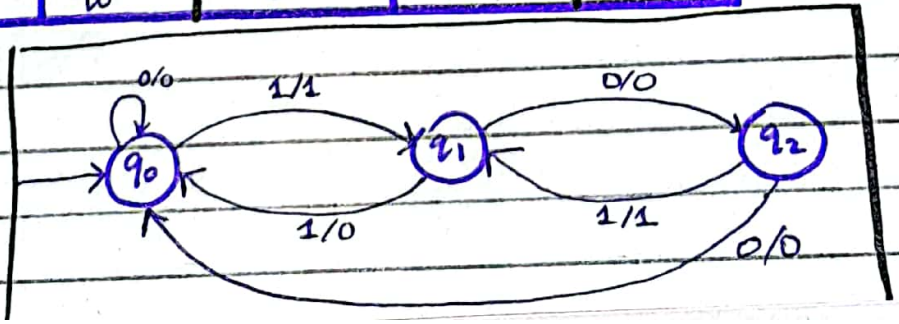


current state	next state		output
	0	1	
q_0	q_0	q_1	0
q_1	q_2	q_0	1
q_2	q_0	q_1	0

Transition table for moore machine

current state	input=0		input=1	
	state	output	state	output
q_0	q_0	0	q_1	1
q_1	q_2	0	q_0	0
q_2	q_0	0	q_1	1

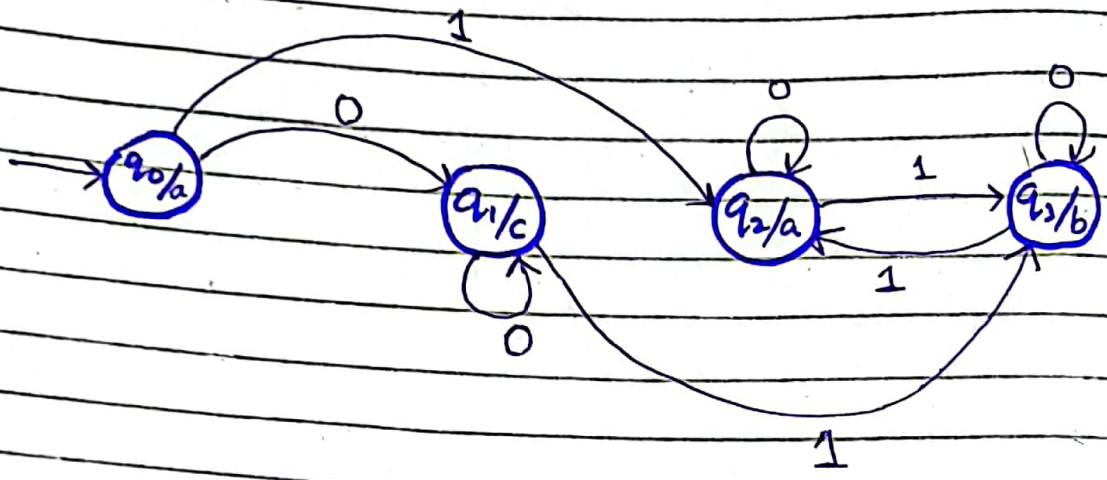
Transition table for mealy machine.



Moore machine

design a moore machine from given transition table

current state	next state		output
	0	1	
q_0	q_1	q_2	a
q_1	q_1	q_3	c
q_2	2_2	q_3	a
q_3	q_3	q_2	b

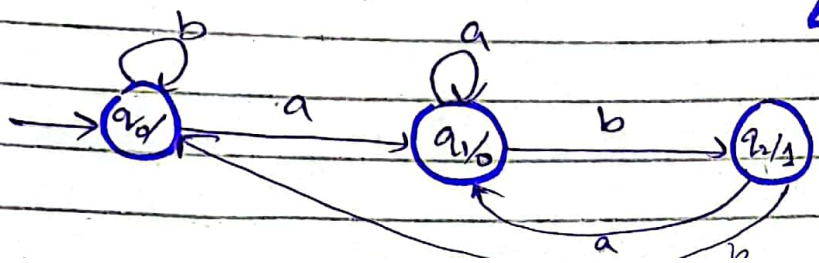


Example:

Construct a Moore machine that takes set of all strings over {a, b} as input & print '1' as output for every occurance of ab as substring.

$\Sigma = \{a, b\}$

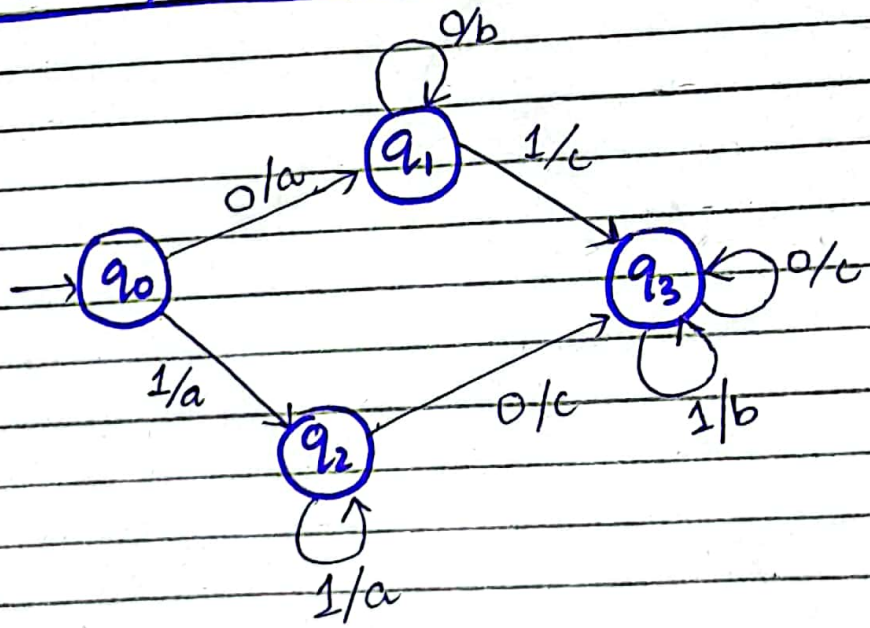
$\Delta = \{0, 1\}$



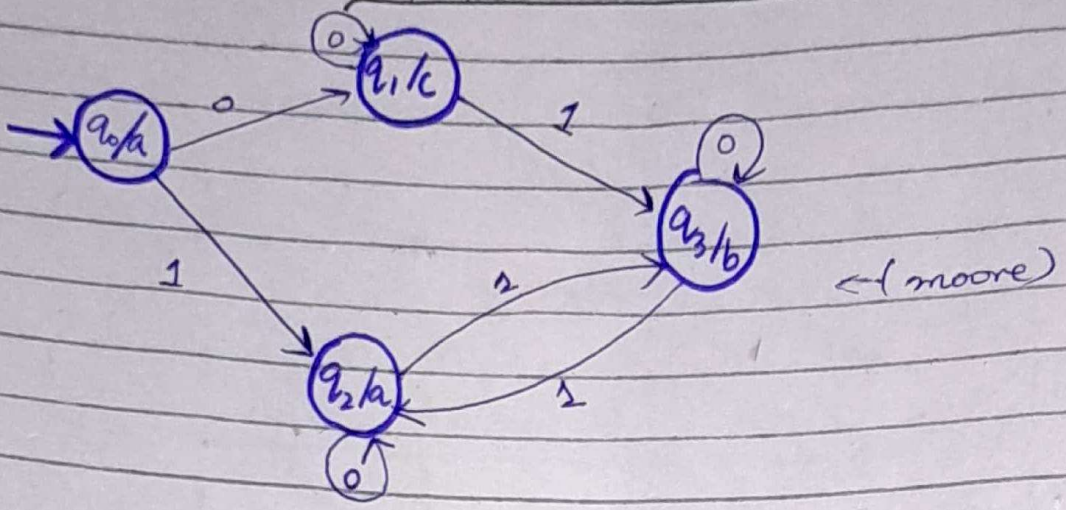
Mealy machine

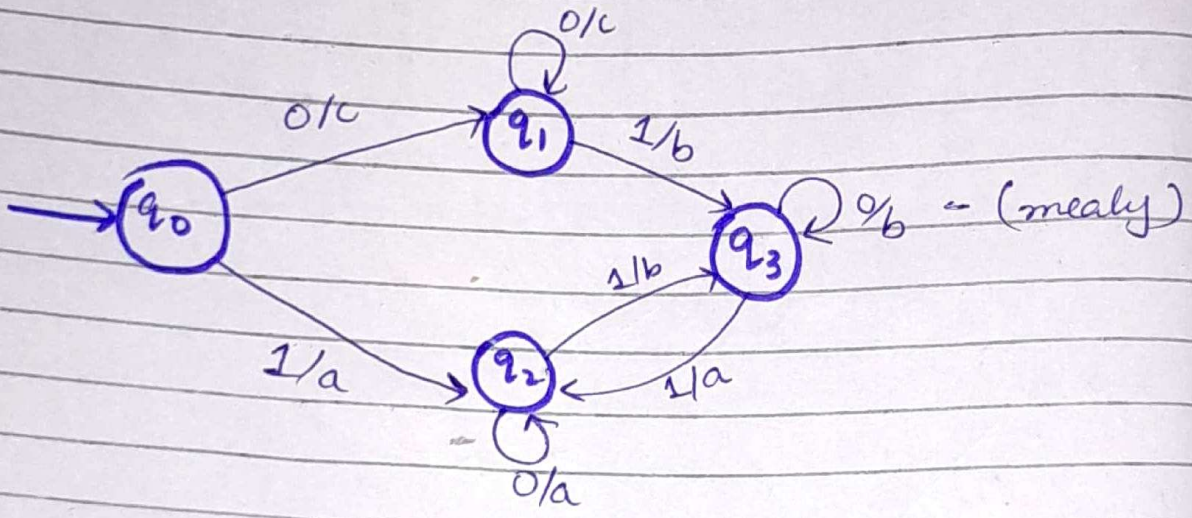
design a Mealy machine from given transition table

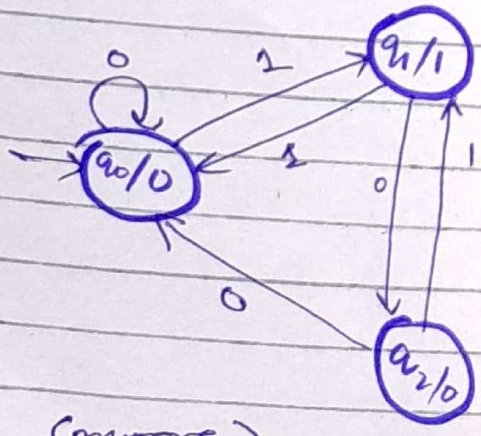
current state	Next State			
	input = 0		input = 1	
	state	output	state	output
q_0	q_1	a	q_2	a
q_1	q_1	b	q_3	c
q_2	q_3	c	q_2	a
q_3	q_3	c	q_3	b

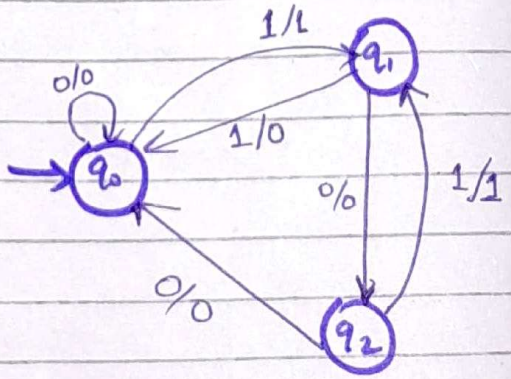


MOORE machine to Mealy

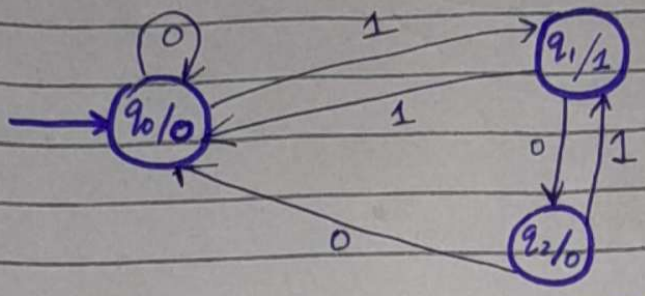






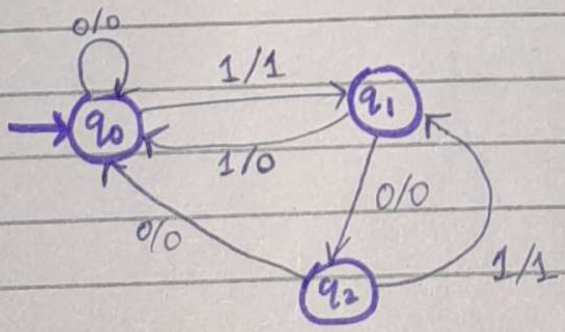


construct a transition table of given moore & convert it into transition table of mealy machine

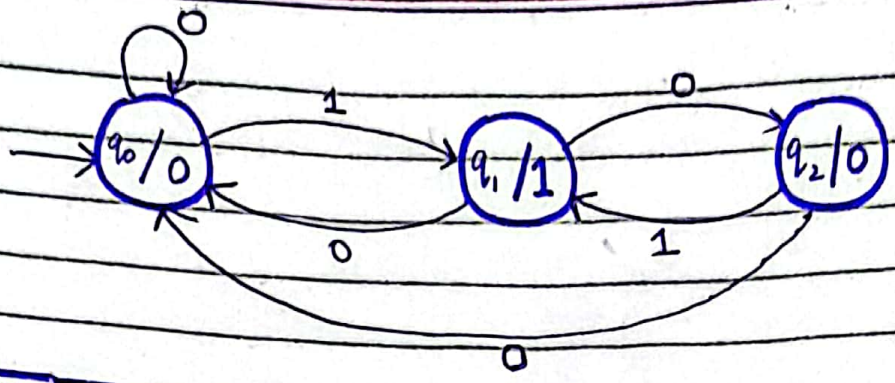


current state	next state		output
	0	1	
q_0	q_0	q_1	0
q_1	q_2	q_0	1
q_2	q_0	q_1	0

current state	input = 0		input = 1	
	next	output	next	output
q_0	q_0	0	q_1	1
q_1	q_2	0	q_0	0
q_2	q_0	0	q_1	1



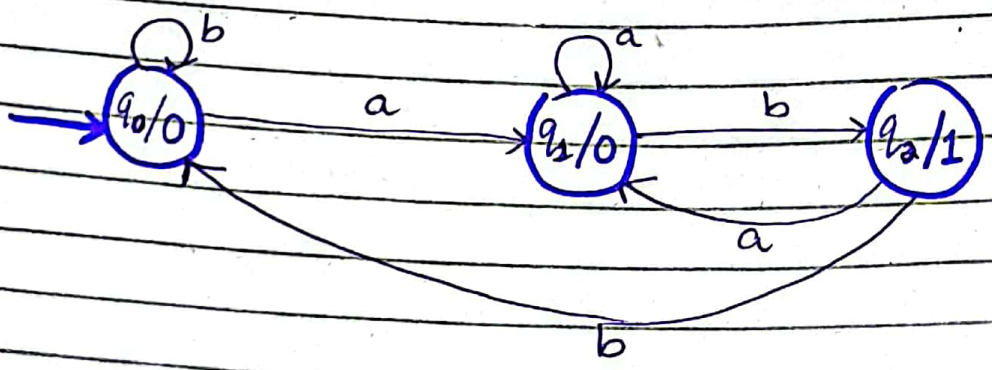
ii)



current state	next		output
	0	1	
q_0	q_0	q_1	0
q_1	q_2	q_0	1
q_2	q_0	q_1	0

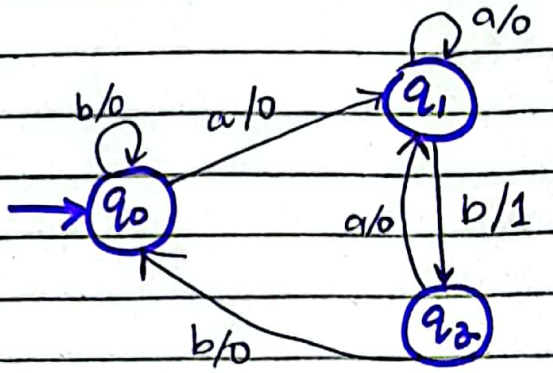
same questions as before

iii)

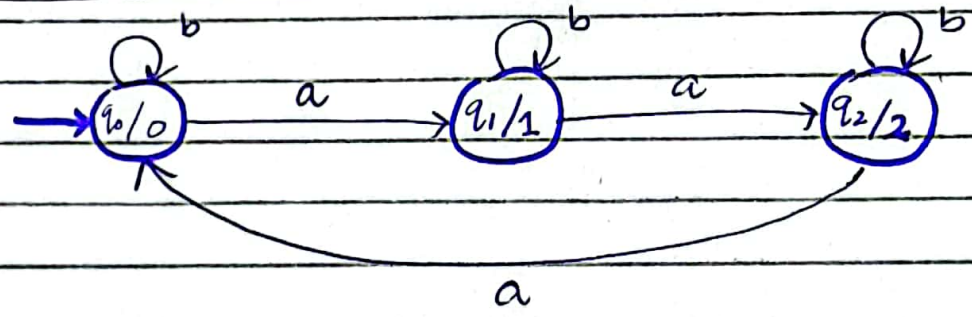


current state	next		output
	a	b	
q_0	q_1	q_0	0
q_1	q_1	q_2	0
q_2	q_1	q_0	1

current state	input=a		input=b	
	next	output	next	output
q_0	q_1	0	q_0	0
q_1	q_1	0	q_2	1
q_2	q_1	0	q_0	0

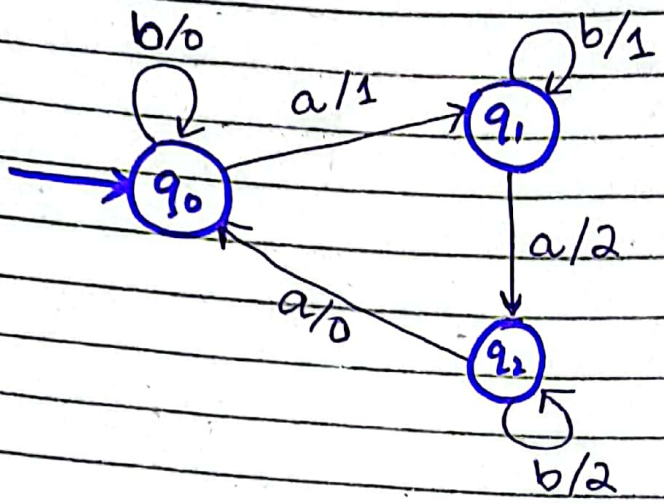


iv)



current state	next		output
	a	b	
q_0	q_1	q_0	0
q_1	q_2	q_1	1
q_2	q_0	q_2	2

current state	input = a		input = b	
	next	output	next	output
q_0	q_1	1	q_0	0
q_1	q_2	2	q_1	1
q_2	q_0	0	q_2	2

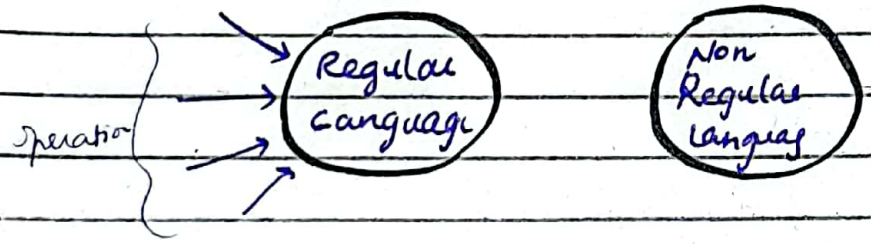


Regular languages & Closure + Decision Properties

Regular language that can be defined by Regular Expression

→ Closure properties
→ Decision properties

cannot be defined by RE is called non regular
e.g. PALINDROME, PRIME



Closure properties of RL

applying operations on RL giving some result & the resultant is also a RL

Properties

→ Union of 2 RL is regular
→ Compliment of RL is also regular
→ Intersection of 2 RL is also regular
→ Closure of RL is also regular (L*)
→ Product / Concatenation of RL is also regular

Union

if L & M are regular languages over Σ then

$L \cup M$ is the language that contain all strings of L & M

"Set of Regular languages is closed under Union"

Set of {some string} closed under {operation name}

proof by RE

→ R.E's

→ F.A's

if L_1 & L_2 are 2 RL then they will have 2 regular expression r_1 & r_2

$L_1 \rightarrow R.E_1$

$L_2 \rightarrow R.E_2$

than

$(R.E_1 + R.E_2)$ is a regular expression that defines the language $(L_1 + L_2)$

hence,

$(L_1 \cup L_2)$ is regular language.

Integer Z = 1, 2

$1 + 2 = 3$ → is also integer

set of integer is closed under addition

$\frac{3}{2} = 1.5$ → not an integer

set of integer is not closed under division.

Example of RL under Union

$L_1 = a^*$ → R.L$_1$

$L_2 = (a+b)$ → R.L$_2$

$L_1 \cup L_2 = a^* \cup (a+b)$

$= a^* + (a+b)$

proof by machines

L_1 : → q_0 —a→ q_1

L_2 : → q_2 —b→ q_3

$L_1 \cup L_2$

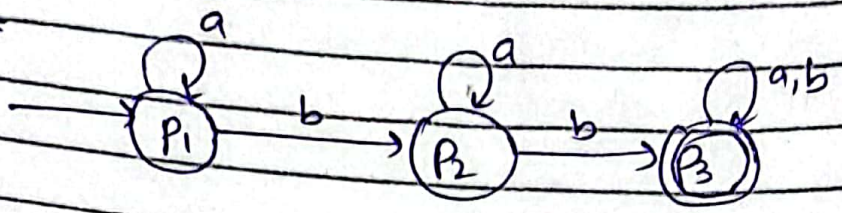
→ q_4 —ε→ q_0 —a→ q_1

q_4 —ε→ q_2 —b→ q_3

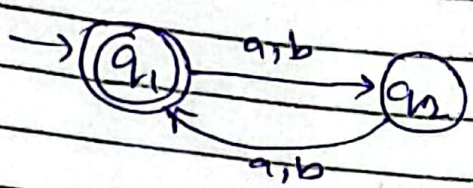
Union by FAs

Formal Proof

FA1

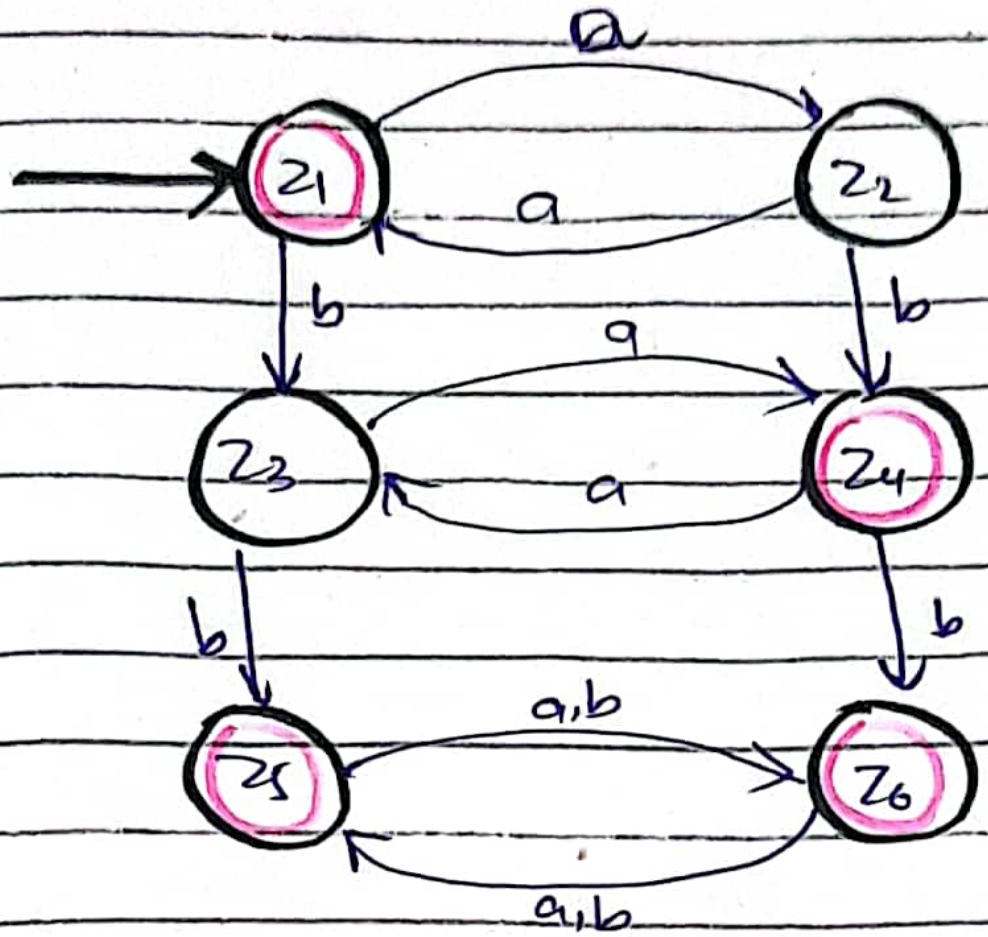


FA2



Note:
Mark all those (as final states) where P3 OR q1 appears anywhere

oldstate	a	b
→ $P_1q_1 = Z_1$	$P_1q_2 = Z_2$	$P_2q_2 = Z_3$
$P_1q_2 = Z_2$	$P_1q_1 = Z_1$	$P_2q_1 = Z_4$
$P_2q_2 = Z_3$	$P_2q_1 = Z_4$	$P_3q_1 = Z_5$
+ $P_3q_1 = Z_4$	$P_2q_2 = Z_3$	$P_3q_2 = Z_6$
+ $P_3q_1 = Z_5$	$P_3q_2 = Z_6$	$P_3q_2 = Z_6$
+ $P_3q_2 = Z_6$	$P_3q_1 = Z_5$	$P_3q_1 = Z_5$



Complement

If L is a regular language over Σ then its complement denoted by L^c consisting all strings from Σ^* that are not words in L

i) by RE

★ $L^c = \Sigma^* - L$

ii) by DFA

$L \rightarrow$ DFA

$\overline{DFA} \rightarrow$ DFA

making all final states nonfinal & nonfinal states final.

★ Example

i) $L^+ = L^* - \{\lambda\}$

ii) $\overline{L} = \Sigma^* - L$

$\overline{\{a, ba\}} = \{\lambda, b, aa, ab, bb, aaa, \ldots\}$

INTERSECTION

if L_1 & L_2 are regular languages
then $L_1 \cap L_2$ is also a regular language

by R.E

De morgan's law

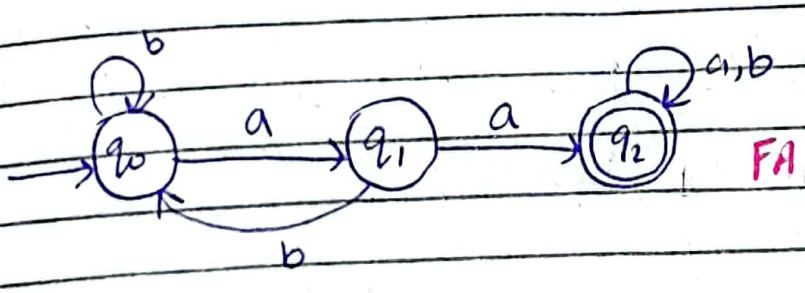
$$L_1 \cap L_2 = L_1^c \cup L_2^c$$

$$= (L_1^c \cup L_2^c)^c$$

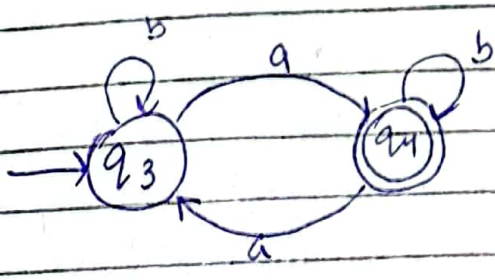
$$= (L_1' + L_2')'$$

$R_1 = (a+b)^* aa (a+b)^*$

$R_2 = b^*(ab^*ab^*)^*$



FA_2

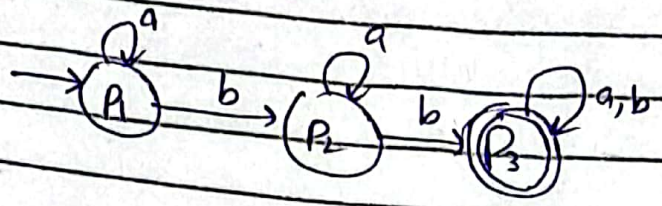


FA_2

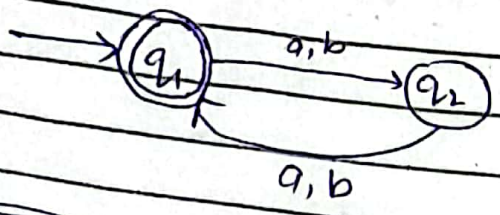
Intersection of FAs

Formal Proof

FA_1 =



FA_2 =



NOTE
mark all those states (as final states) where both P3 and q1 appears together because it is intersection.

1 step

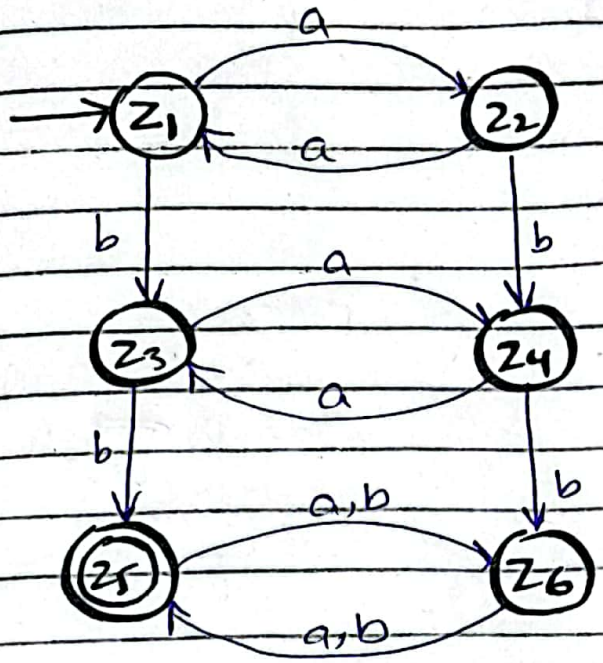
make trasition table for both FAs combined

old state	a	b
→ $P_1q_1 = z_1$	$P_1q_2 = z_2$	$P_2q_2 = z_3$
$P_1q_2 = z_2$	$P_1q_1 = z_4$	$P_2q_1 = z_4$
$P_2q_2 = z_3$	$P_2q_1 = z_4$	$P_3q_1 = z_5$
$P_2q_1 = z_4$	$P_2q_2 = z_3$	$P_3q_2 = z_6$
$P_3q_1 = z_5$	$P_3q_2 = z_6$	$P_3q_2 = z_6$
$P_3q_2 = z_6$	$P_3q_1 = z_5$	$P_3q_1 = z_5$

After two page

→ here

$FA_1 \cap FA_2$.



Set of RL is closed under intersection.

Closure : Kleene Closure

If L is a RE then L* consists of all strings formed by the concatenation of arbitrary many factors of L

$L_1 \rightarrow L_1^* \rightarrow$ regular language

by R.E

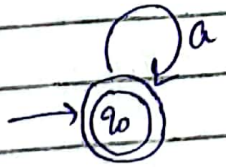
i) $L_1 = a$

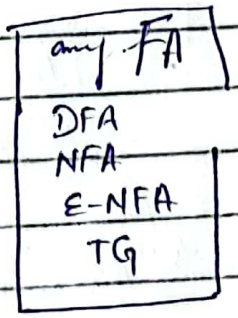
$L_1^* = a^*$

ii) $L_1 = a^*b$

$L_1^* = (a^*b)^*$

by F.A





Concatenation

If L_1 & L_2 are two RL then $L_1 \cdot L_2$ contains all words formed by concatenation of a word from L_1 with a word from L_2

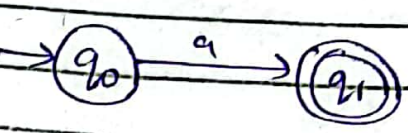
by RE

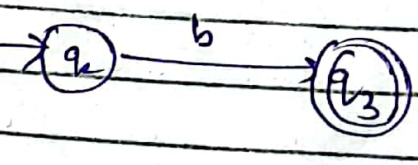
$L_1 = RE_1$

$L_2 = RE_2$

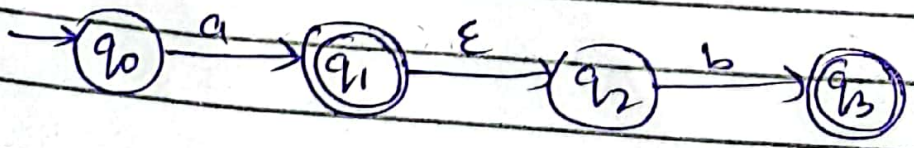
$L_1 \cdot L_2 = RE_1 \cdot RE_2$

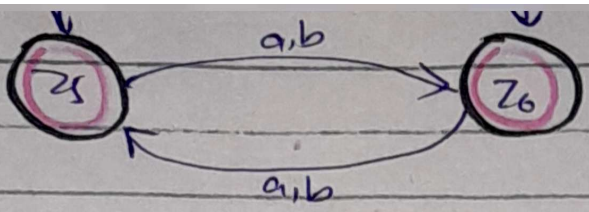
by FA

$FA_1 (L_1) =$ 

$FA_2 (L_2) =$ 

$FA_1 \cdot FA_2 =$





NON-REGULAR LANGUAGES

Pumping Lemma

→ a test to prove that a language is non regular language

machine that is used prove certain languages as non-regular

→ powerful machines.

lemma

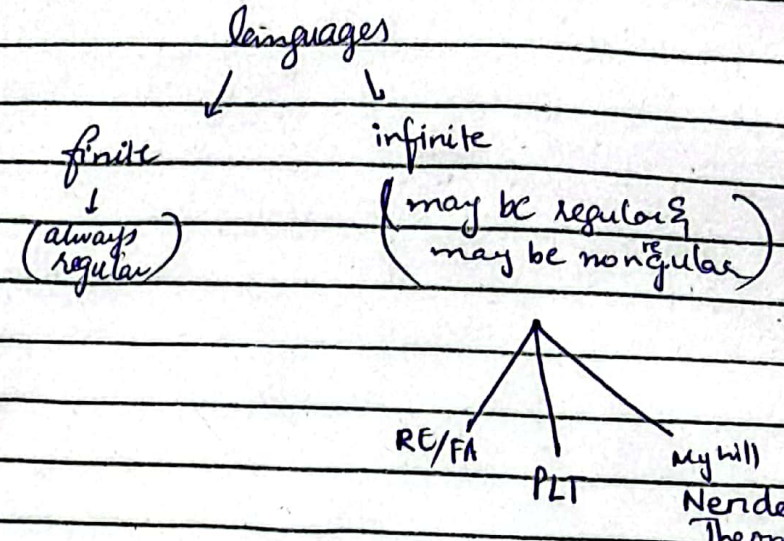
small helper rule used to prove something bigger

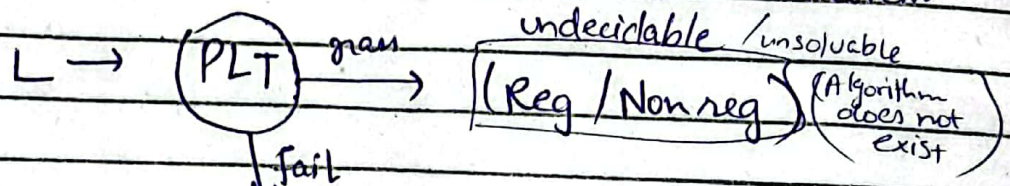
→ help us show that a language is not regular

→ It provides conditions that a string must satisfy if those conditions are violated (negative test) then the language is non-regular.

Pumping lemma Test (PLT)

negative test





Decidable → (100%) non-regular

↓

(Algorithm can be written)

ALGORITHM

(Q) Prove that language L is Non-regular

Proof steps:-

(1) Suppose that language L is regular

(2) Then we can make FA of L

↳ suppose the FA of language is of N states

(3) Take a string / word 'w' (from L) of length greater than

number of the states (N). i.e. $w > N$

(4) Divide this word into Three part x, y & z

such that $len(x) + len(y) < N$
or $len(xy) < N$

(5) 'y' can't be null
'x' & 'z' can be null
$len(y) \geq 1$

(6) pump 'y' and decide that language L is Non regular

~~Example~~

$n+1$ ... $2^{n+1}-1$

servation
We can't go
valid anymore.

Example

$L = a^n b^n$

1. Suppose $L = a^n b^n$ regular
2. Suppose that it's FA is of 10 states (i.e N = 8)
3. Take a string 'w' is such that $w > N$
4. $w = \underbrace{aaaaa}_{x}\ \underbrace{bbbbb}_{y}\ \underbrace{}_{z}$ $(10 > 8)$

len (x) = 2
len(y) = 3
len (x+y) = 5
len (x+y) = 5 ≤ N
(5) < 8

5. This condition is met
len (y) = 3
len(y) ≥ 1 (True)

6. Pump(y) = x yyz
aa ↙ ↓ ↓ ↘
aaa aaa bbbbb

does → aaaaaaabbbbbb ∈ $L_{(n>0)}(a^n b^n)$

pump(y) → aa aaaaaaaaa bbbbb

4. Repeat

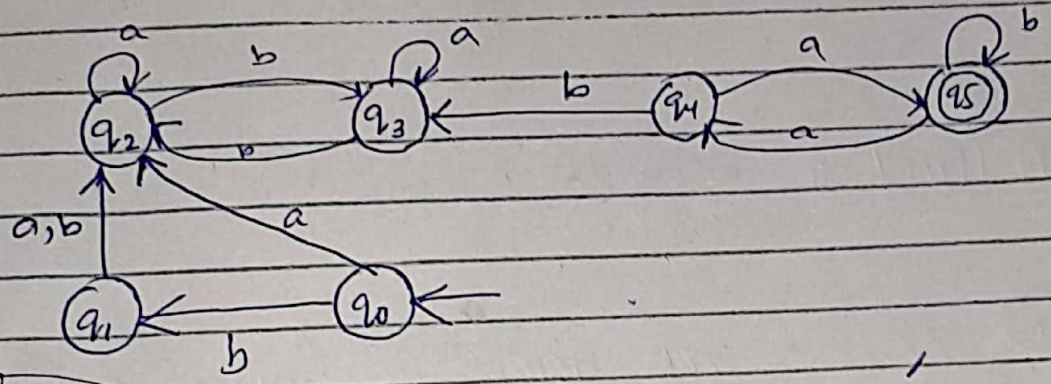
5. Observation
We can
valid a

Decidability → jiska algorithm likha jaskta ha

→ emptiness
→ equivalence
→ ~~finiteness~~ finiteness

Emptiness / Non-Emptiness

whether an RE/FA accepts any string or not?



proof steps

1. mark the initial state
2. mark the states that are connected with initial state
3. remove the edges that connect the initial states with those states
4. Repeat Step 3 where applicable
5. **Observation** We can't go further if step 3/4 are not valid any more.

(6) If the final state is not marked then the FA doesn't accept any string } → emptiness

(7) if some of the final states is marked then the FA does accept any string.

Method 2 Regular Expression

(1) remove all asterik(s) from the RE

(2) Try to generate any string from that RE

$$(a+b)^* (a+b) (a+b)^*$$

$$(a+b)(a+b)(a+b)$$

generate string aba, aab, bbb

yes → Non emptiness is proved.

(2) Finiteness / Infiteness

* whether a RE/FA is of a finite or infinite language?

Case 1:-

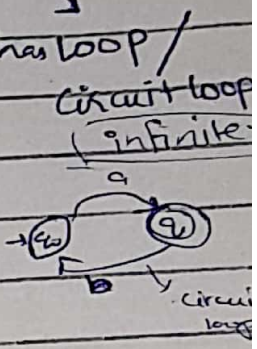
R.E

- No kleen Star → is finite language
  - (a+b)b(a+b)
- Has kleen Star
  - finite → ε^*
  - infinite → a^*b^*, ab^*,

Case 2:-

F.A

- No loop / No circuit loop → finite
- has loop / circuit loop → infinite



(3) Equivalence / Equality

We need to find this expression

$$(L_1 \cap L_2^c) \cup (L_1^c \cap L_2)$$

RE

Check if it accepts any string or not.

Generate (L1 & L2 are not equal)

does not generate (Both L1 & L2 are equal.)